

LAB 1

TITLE: Process Creation using fork() System Call in C.

OBJECTIVE: To understand and implement process creation using the fork() system call, and to demonstrate how a parent and child process operate independently.

THEORY:

The fork() system call is used to create a **new process** by duplicating the calling process. The newly created process is called the **child process**, and the original is called the **parent process**.

fork() Characteristics:

- It returns:
 - 0 to the child process
 - A positive value (PID of child) to the parent process
 - -1 if the process creation fails
- After calling fork(), both parent and child execute the same code but can behave differently using if (pid == 0) and else condition.

- The wait() function makes the parent wait until the child process terminates.

Key Functions Used:

- fork() – to create a new process
- getpid() – to get current process ID
- getppid() – to get parent process ID
- wait() – to make parent wait for child
- sleep() – to simulate delay

Algorithm:

1. Start
2. Declare pid variable of type pid_t
3. Call fork() to create a new process
4. Check if fork() returned:
 - < 0: Print error and exit
 - 0: It is the child process. Print child details and simulate work with sleep()
 - > 0: It is the parent process. Print parent details and wait for the child to finish
5. Exit both processes

PROGRAMS

```
#include <stdio.h>
#include <unistd.h>    // Required for fork() and sleep()
#include <sys/types.h> // Required for pid_t
#include <sys/wait.h>  // Required for wait()
```

```
int main() {
    pid_t pid; // Process ID variable
```

```
    // Create a new process
    pid = fork();
```

```
    // Check for fork error
    if (pid < 0) {
        fprintf(stderr, "Fork failed\n");
        return 1;
    }
```

```
    // Child process
```

```
    if (pid == 0) {
        printf("Child process:\n");
        printf(" PID: %d\n", getpid());
        printf(" Parent PID: %d\n", getppid());
        printf(" Child is sleeping for 2\n");
        seconds...\n");
        sleep(2); // Simulate some work
        printf(" Child exiting\n");
    }
    // Parent process
    else {
        printf("Parent process:\n");
        printf(" PID: %d\n", getpid());
        printf(" Child PID: %d\n", pid);
        printf(" Parent waiting for child...\n");
        wait(NULL); // Wait for child to finish
        printf(" Parent exiting\n");
    }
}
```

```
return 0;
}

```

Output:

```
Parent process:
  PID: 3421
  Child PID: 3422
  Parent waiting for child...
Child process:
  PID: 3422
  Parent PID: 3421
  Child is sleeping for 2 seconds...
  Child exiting
  Parent exiting
```

RESULTS:

The program successfully demonstrates process creation using the `fork()` system call. A parent process creates a child process, waits for it to finish, and then exits.

CONCLUSION:

This lab provided hands-on experience with process management in OS, particularly using the `fork()` system call. It showed how processes are created and how a parent can manage child execution using `wait()`.